

CE67 - 13

Automatic License Plate Recognition Service



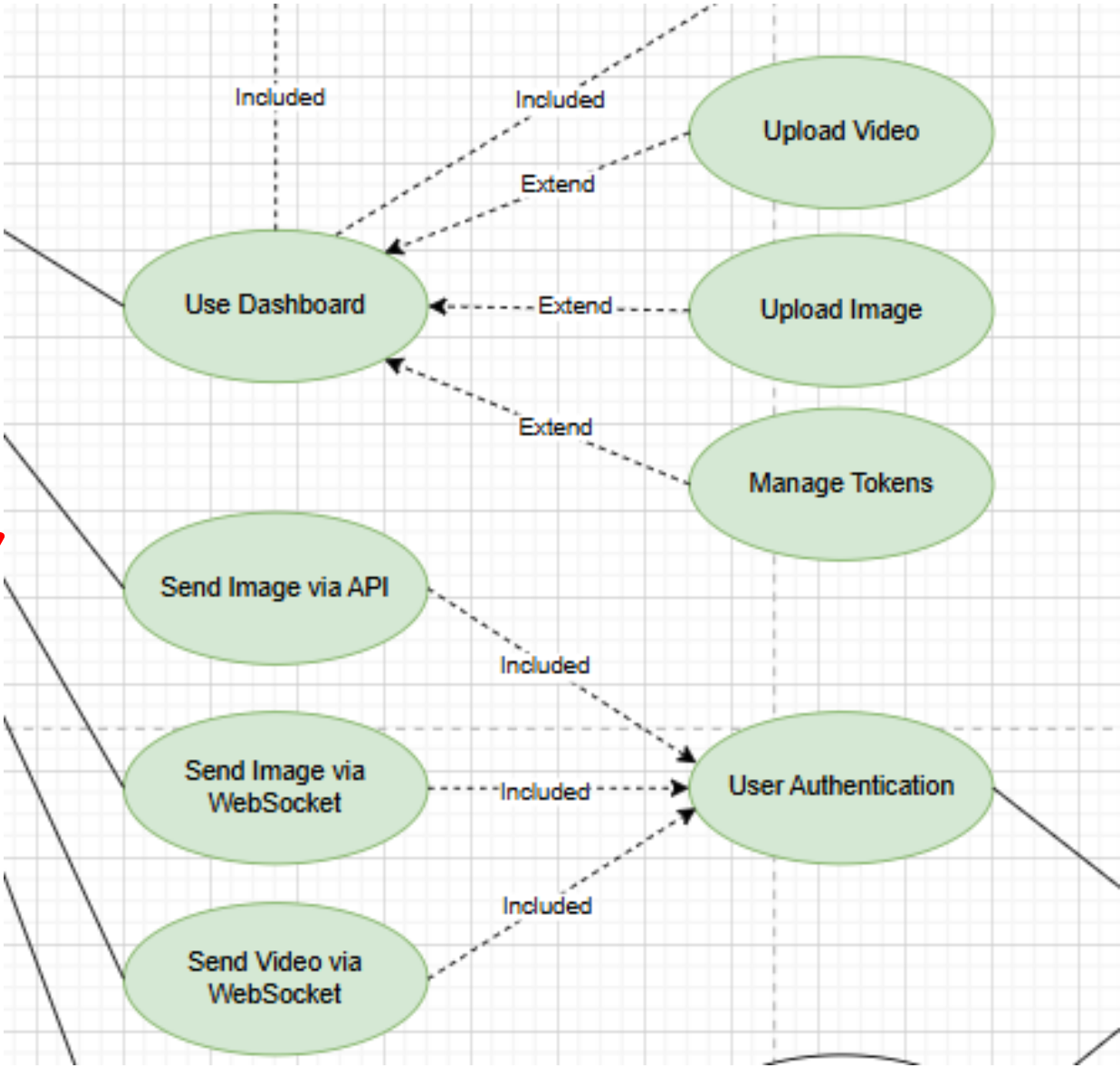
ระบบให้บริการอ่านป้ายทะเบียนแบบอัตโนมัติ

Goals

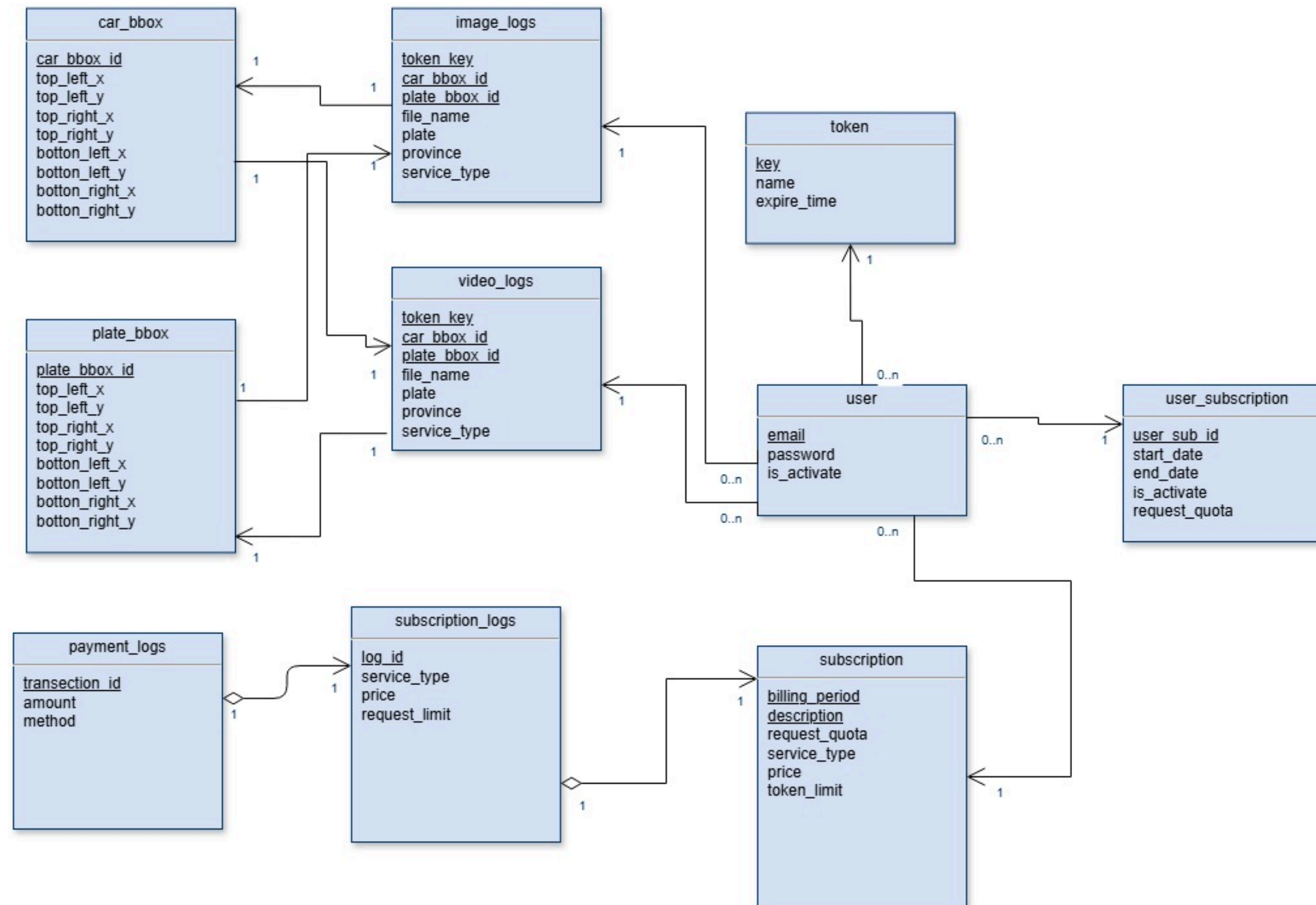
- สร้าง Machine Learning Model ขึ้นมาใหม่ ที่สามารถอ่านป้ายทะเบียนรถยนต์ ทั้งรูปแบบปกติ และรูปแบบพิเศษได้ ที่มีความแม่นยำอย่างน้อย 95 %
- พัฒนาระบบ Subscription Service ให้ผู้ใช้สามารถใช้งาน ผ่านอินเทอร์เน็ตได้ โดยไม่ต้อง ติดตั้ง Software เพิ่มเติม



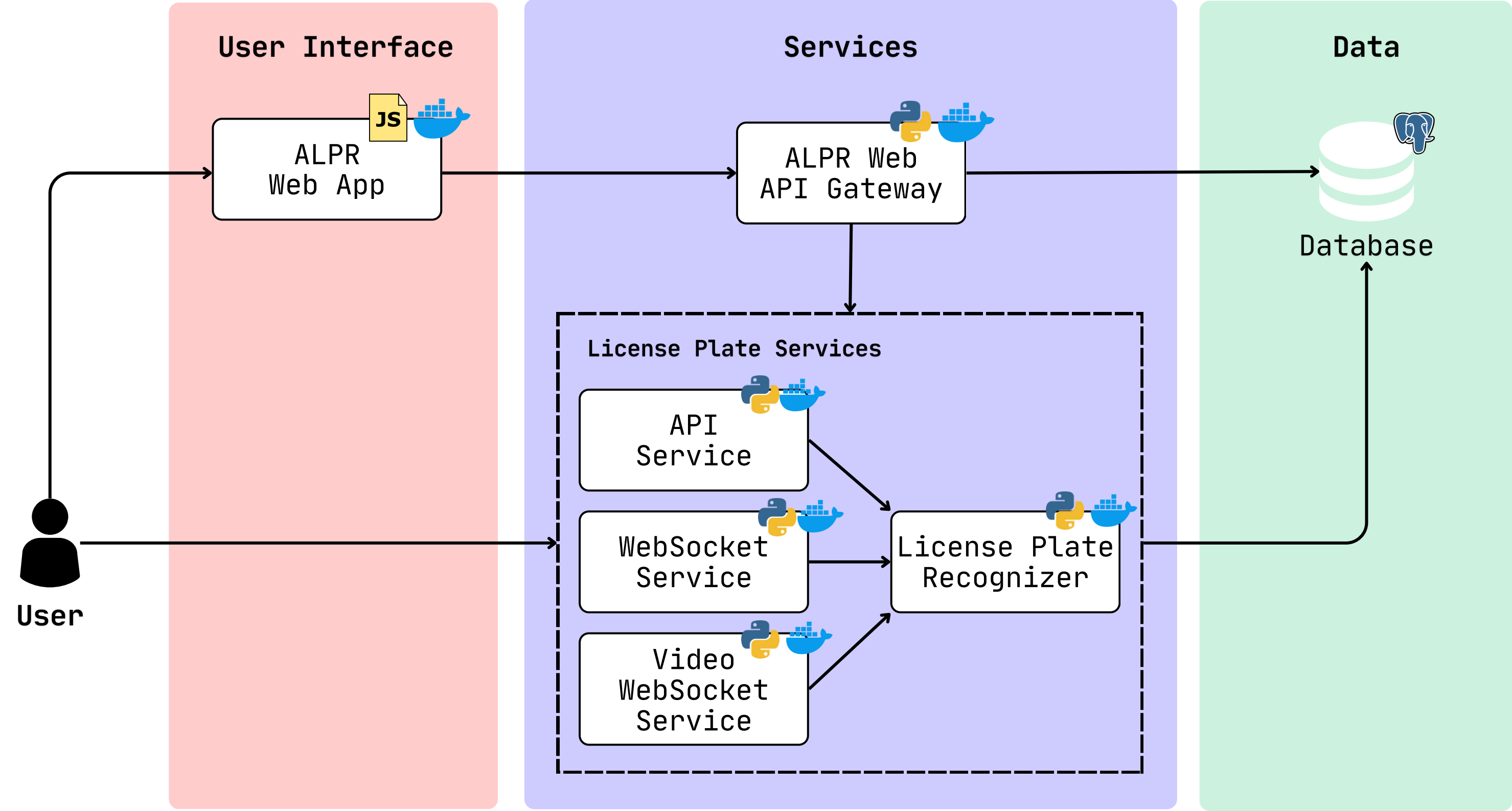
Use Case Diagram



Relational Database

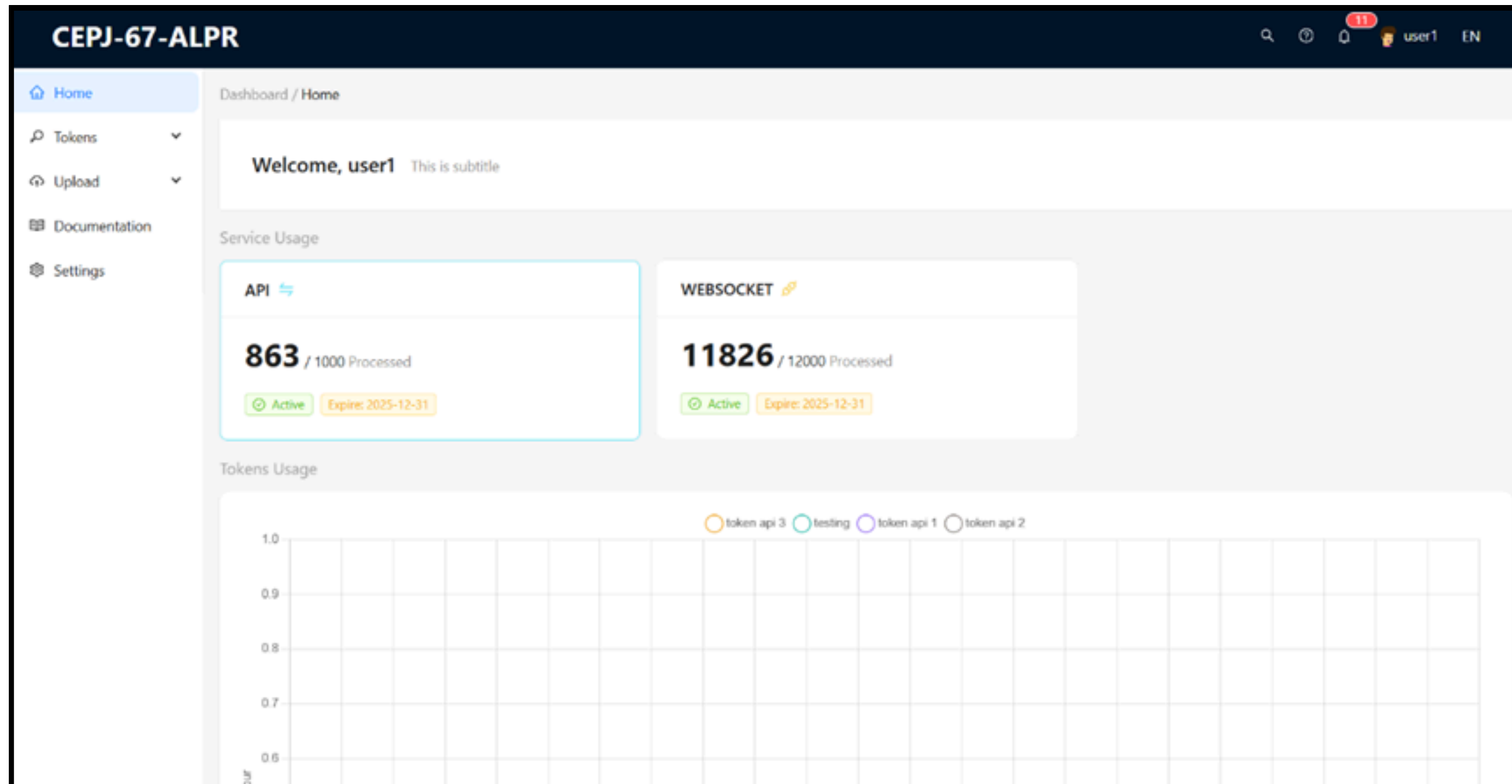


System Architecture Diagram



ALPR Web App - Dashboard

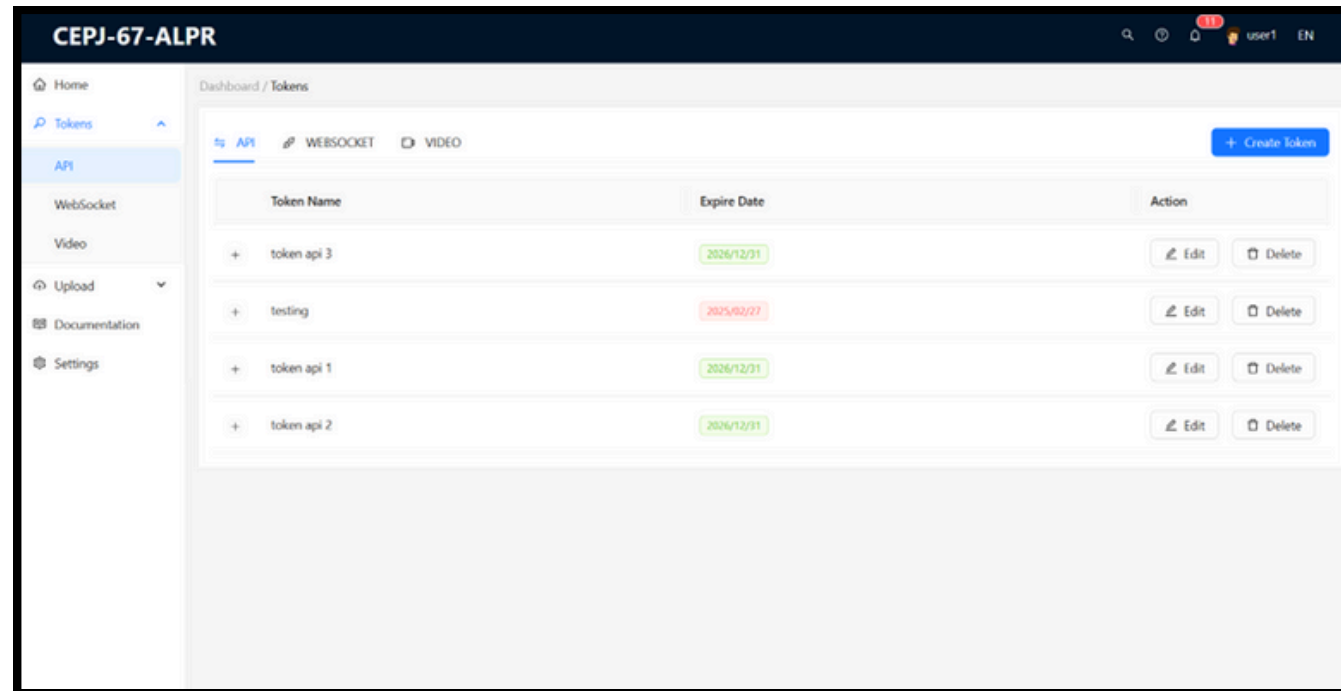
6



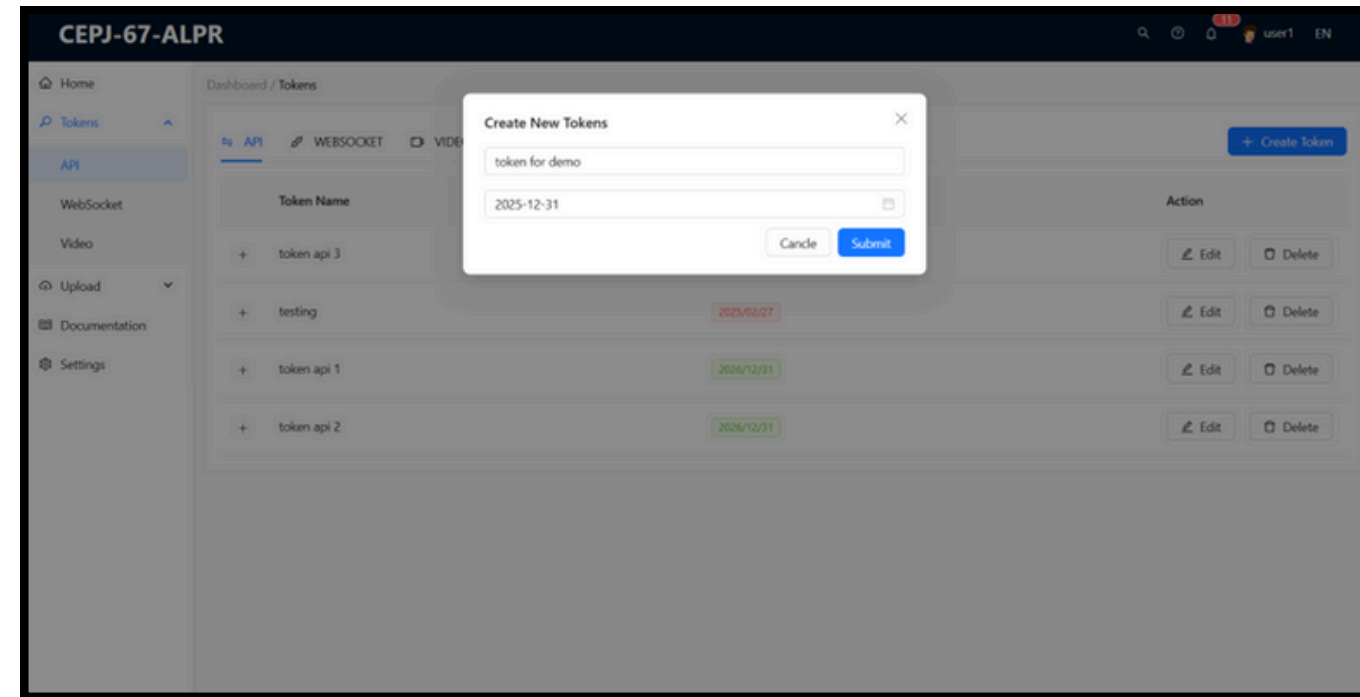
แสดงข้อมูลการใช้งานของผู้ใช้

ALPR Web App - Manage Tokens

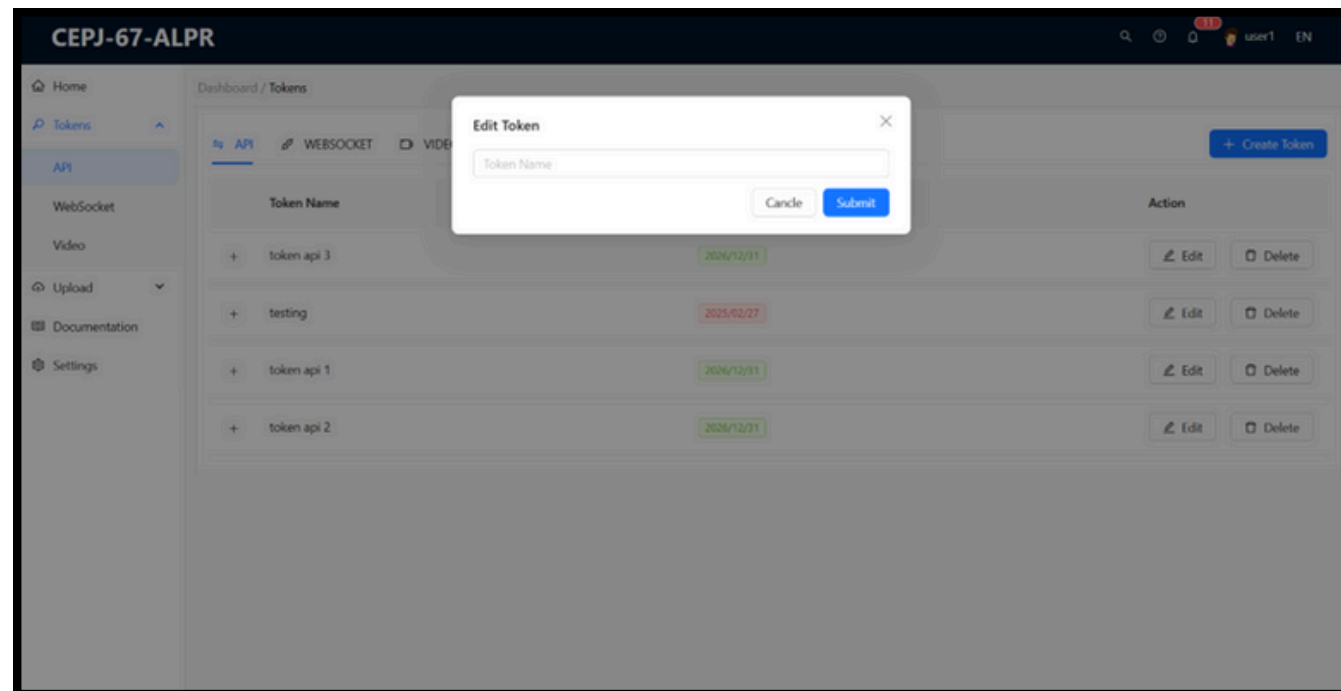
7



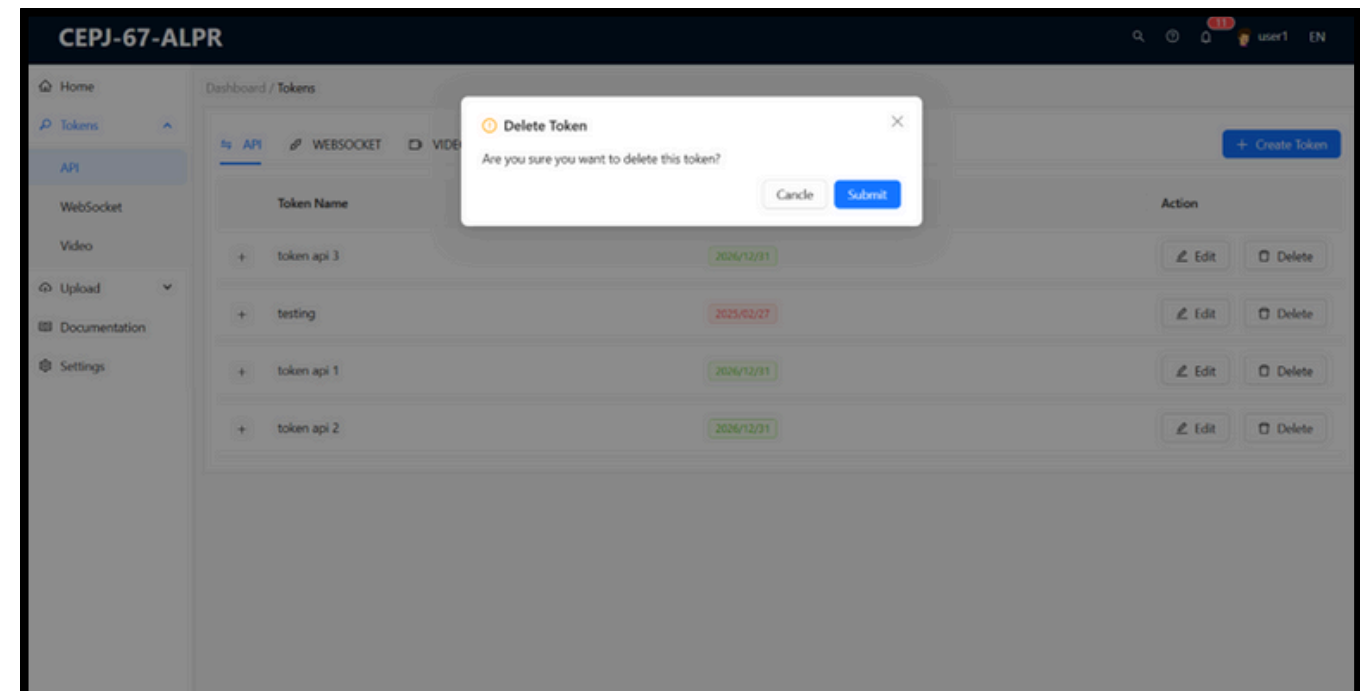
แสดงข้อมูล Token ทั้งหมด



สร้าง Token ใหม่



แก้ไขข้อมูล Token



ลบ Token

ALPR Web App - Upload Image

8

CEPJ-67-ALPR

Home

Tokens

Upload

Image

Video

Documentation

Settings

Dashboard / Upload

Upload Image

JPG, JPEG, PNG

File Size: 0.00 / 50 MB

token api 3

Submit

Process Result

Car Image	Plate Image	Plate ID	Province	Time Stamp
		กข 5928	กรุงเทพมหานคร	2025-02-28 15:16:50

ALPR Web App - Upload Video

CEPJ-67-ALPR

🔍

🕒

🔔11

👤 jonhdoe

EN

🏠 Home

🔑 Tokens ▾

📶 Upload ▲

🖼️ Image

📺 Video

📖 Documentation

⚙️ Settings

Dashboard / Upload

📶

Click or drag video file to upload

MP4, AVI, MKV

token for video service ... ▾

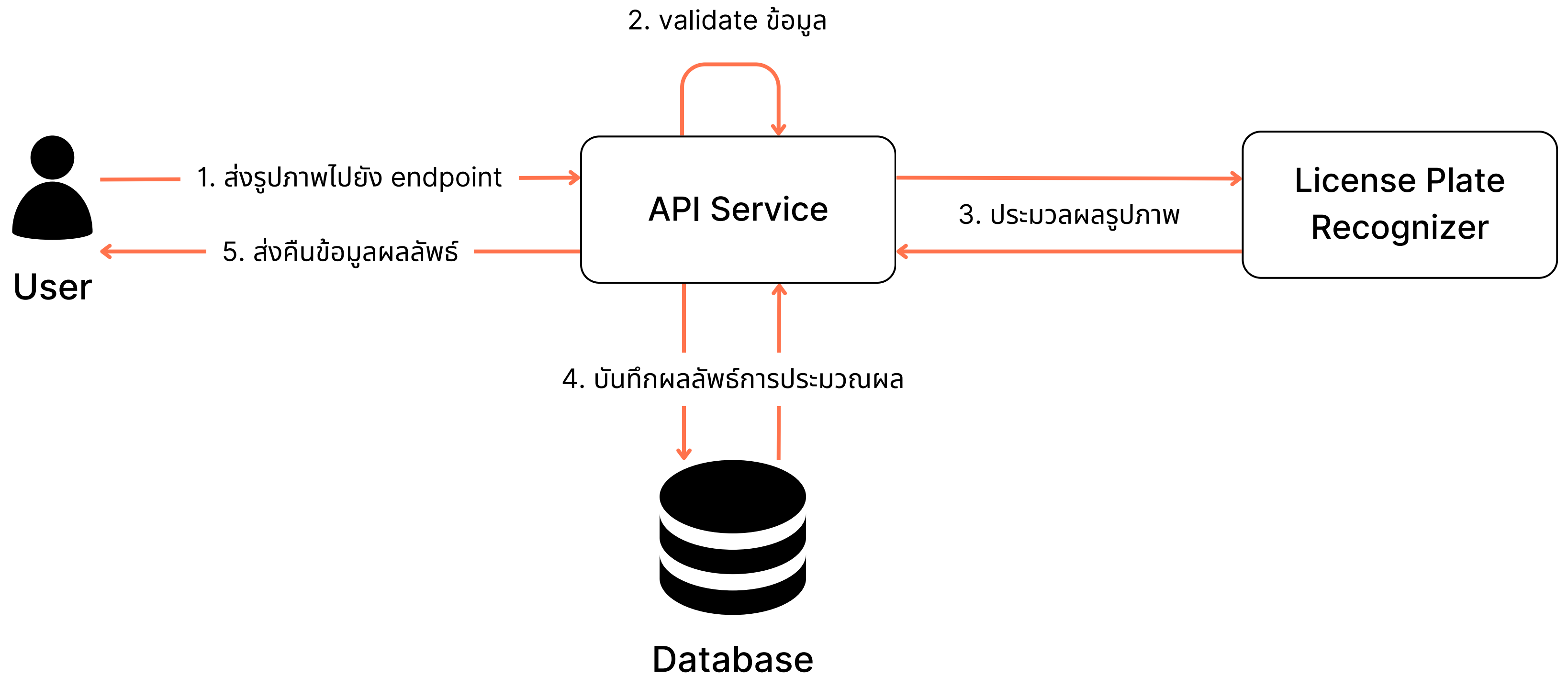
Submit

Process Result

Car Image	Plate Image	Plate ID	Province	Time Stamp
		Not Found	Not Found	2025-02-28 15:49:43
		Not Found	Not Found	2025-02-28 15:49:55
				

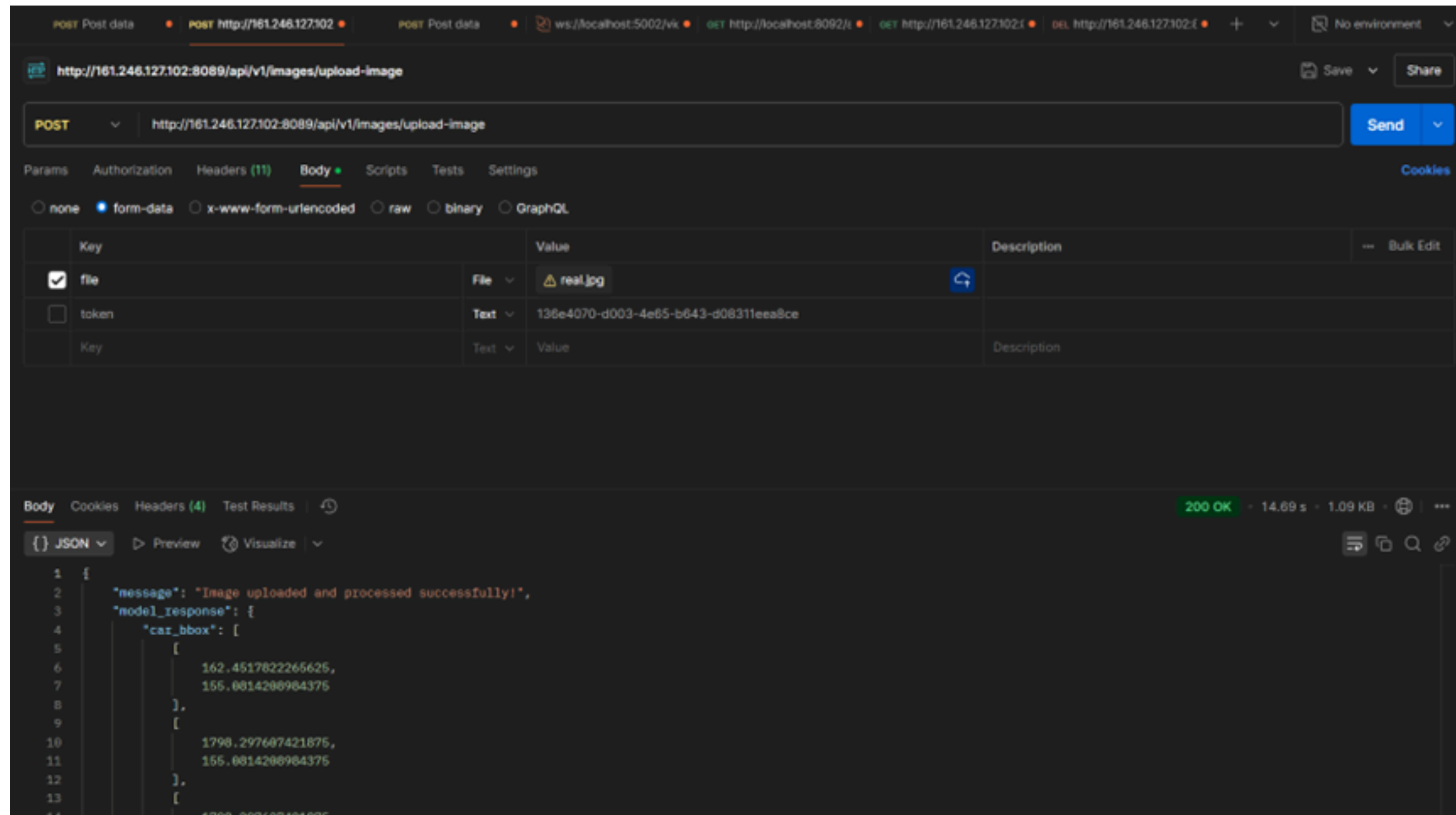
API Service - Send Image via API

10



API Service - Send Image via API

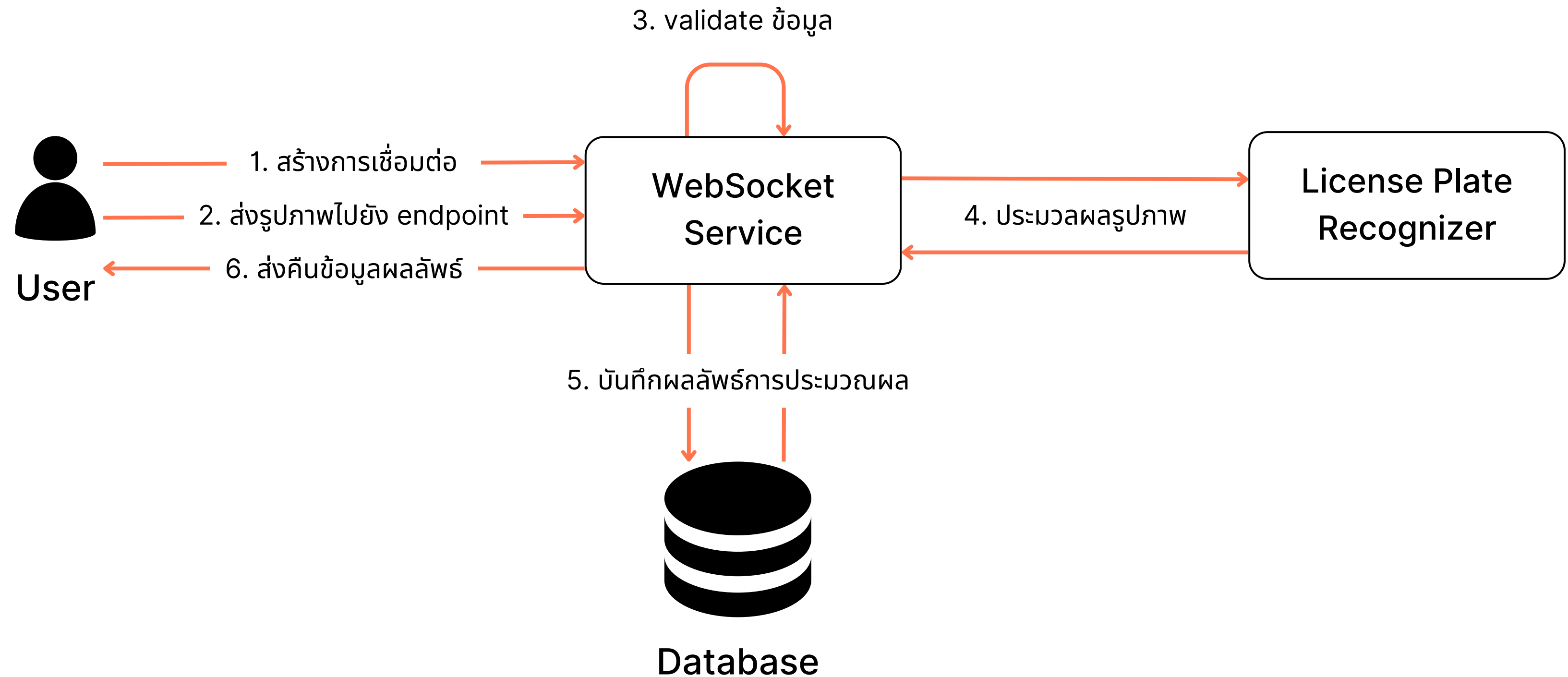
11



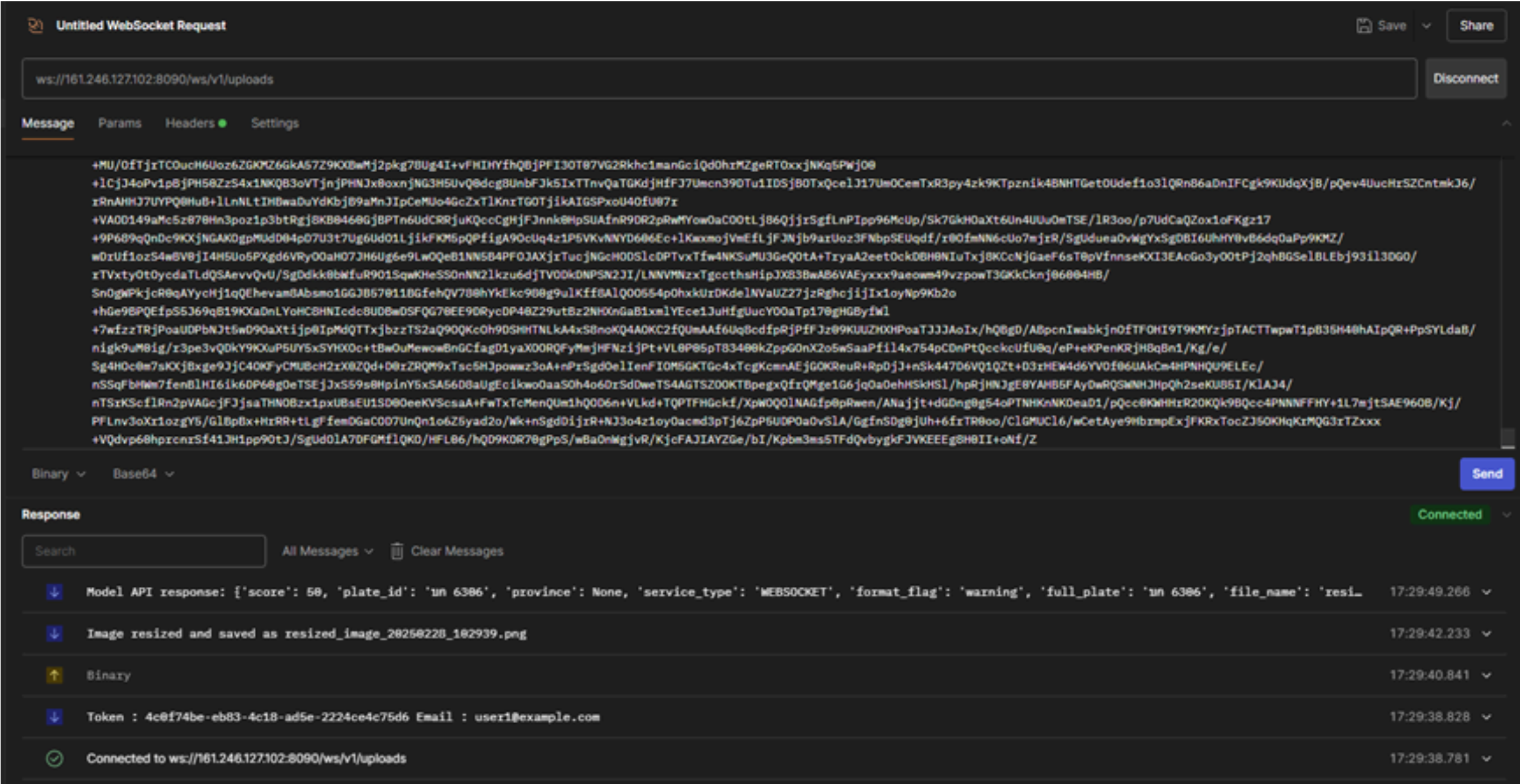
การทดสอบ API โดยใช้ Postman

WebSocket Service - Send Image via WebSocket

12



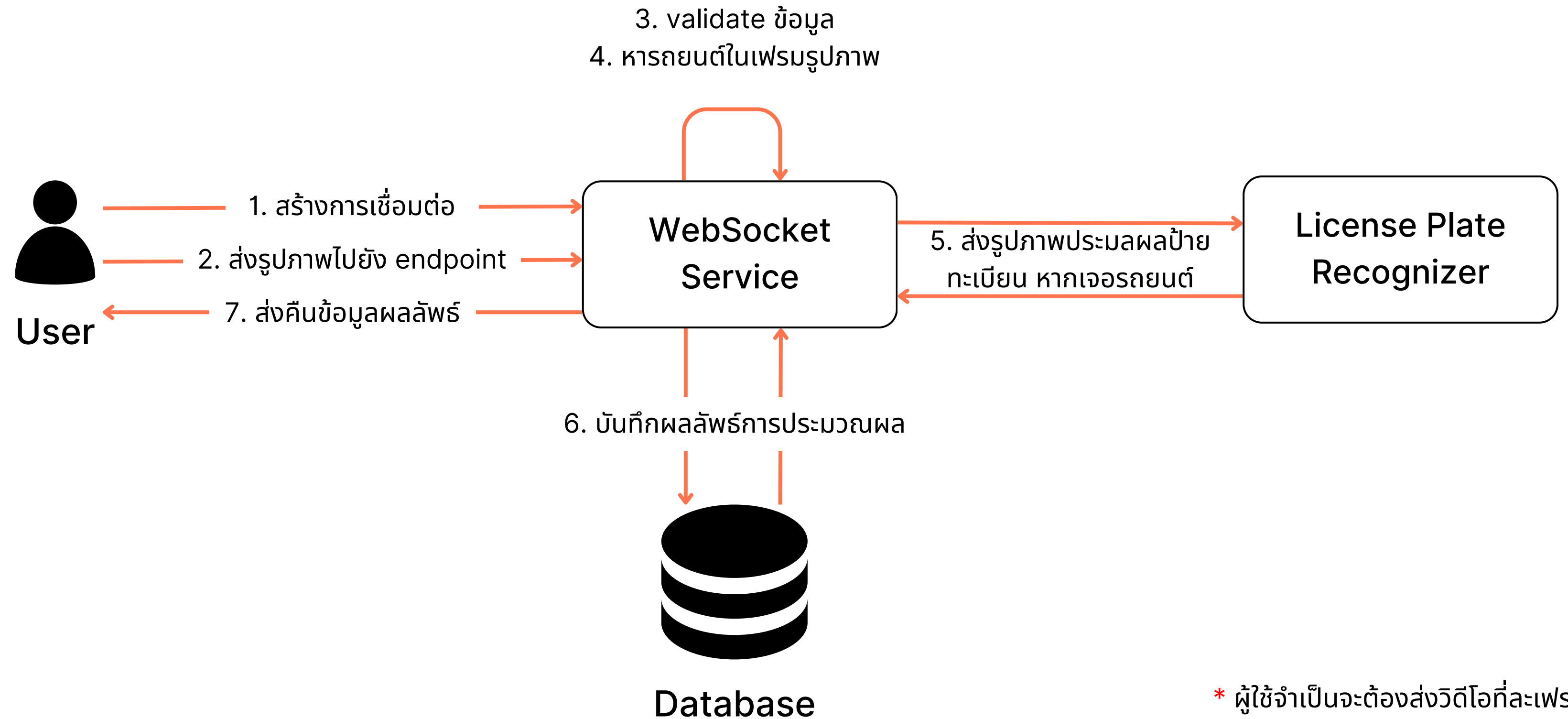
WebSocket Service - Send Image via WebSocket



การทดสอบ WebSocket โดยใช้ Postman

Video WebSocket Service - Send Video via WebSocket

14



Video WebSocket Service - Send Video via WebSocket

15

```
1  async def send_video_frames(websocket, video_path: str):
2      cap = cv2.VideoCapture(video_path)
3
4      if not cap.isOpened():
5          logger.error(f"Failed to open video file: {video_path}")
6          return
7
8      frame_count = 0
9
10     try:
11         while True:
12             ret, frame = cap.read()
13
14             if not ret:
15                 logger.info("No more frames to read. Exiting.")
16                 break
17
18             _, buffer = cv2.imencode('.jpg', frame)
19             frame_bytes = buffer.tobytes()
20
21             await websocket.send(frame_bytes)
22             logger.info(f"Sent frame {frame_count} (size: {len(frame_bytes)} bytes)")
23
24             frame_count += 1
25
26             await asyncio.sleep(1 / 300) # add delay
27
28     except Exception as e:
29         logger.error(f"Error in send_video_frames: {e}")
30     finally:
31         cap.release()
32
```

```
1  async def receive_server_responses(websocket):
2      try:
3          while True:
4              response = await websocket.recv()
5
6              try:
7                  logger.info(response)
8              except Exception as e:
9                  logger.error(f"Error decoding image: {e}")
10
11     except websockets.exceptions.ConnectionClosed as e:
12         logger.error(f"WebSocket connection closed: {e}")
13     except Exception as e:
14         logger.error(f"Error in receive_server_responses: {e}")
```

จำลองการส่งข้อมูลจากระบบของผู้ใช้ด้วย Script Python

Video WebSocket Service - Send Video via WebSocket

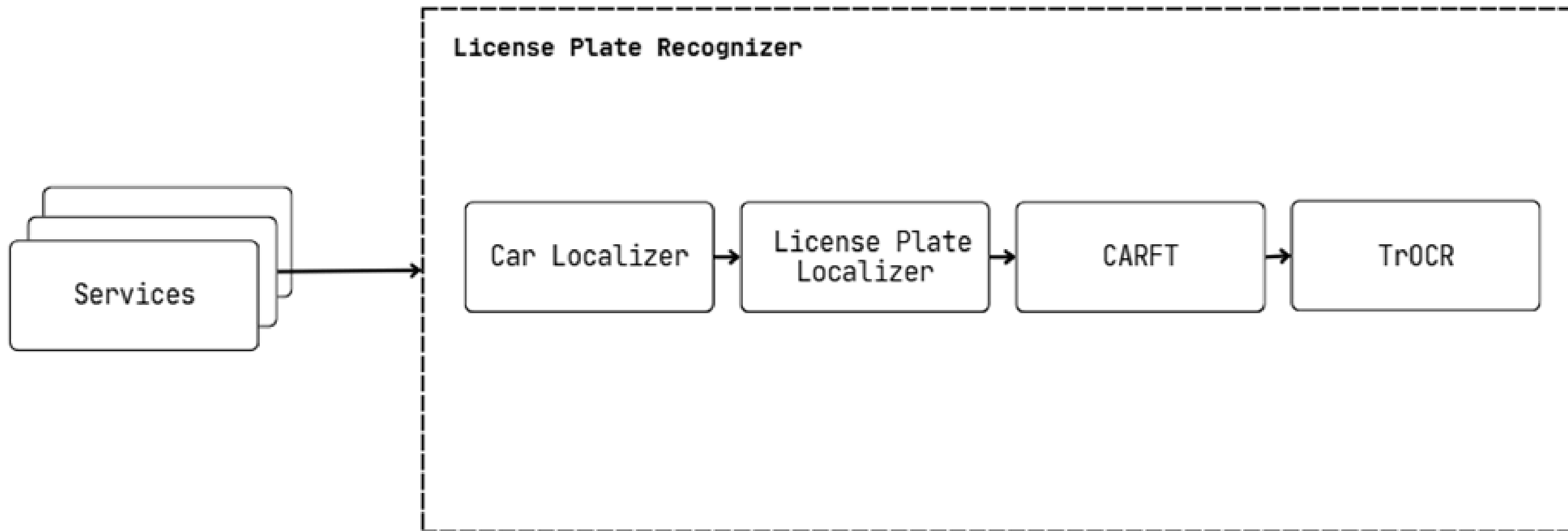
16

```
1  async def main():
2      try:
3          async with websockets.connect(WS_URL) as websocket:
4              logger.info(f"Connected to WebSocket server at {WS_URL}")
5
6              send_task = asyncio.create_task(send_video_frames(websocket, VIDEO_PATH))
7              receive_task = asyncio.create_task(receive_server_responses(websocket))
8
9              await asyncio.gather(send_task, receive_task)
10
11     except (ConnectionClosedError, ConnectionClosedOK) as e:
12         if e.code == 4001:
13             logger.error(f"Connection closed: Invalid token - {e.reason}")
14         elif e.code == 1011:
15             logger.error(f"Connection closed: Internal server error - {e.reason}")
16         else:
17             logger.error(f"Connection closed: {e.code} - {e.reason}")
18     except Exception as e:
19         logger.error(f"Unexpected error: {e}")
20
21 if __name__ == "__main__":
22     asyncio.run(main())
23
```

จำลองการส่งข้อมูลจากระบบของผู้ใช้ด้วย Script Python

License Plate Recognizer

17



ผลลัพธ์จากการประมวลผลป้ายทะเบียน (JSON response)

18

car_bbox: bounding box ของรถยนต์
plate_bbox: bounding box ของป้าย
text_bbox_list: bounding box ของตัวอักษร แต่ละบรรทัด
plate_id: เลขทะเบียน
province: จังหวัด
full_plate: ป้ายทะเบียนเต็ม
message: คำอธิบายผลลัพธ์การประมวลผล
image: รูปภาพ raw bytes

Issue model

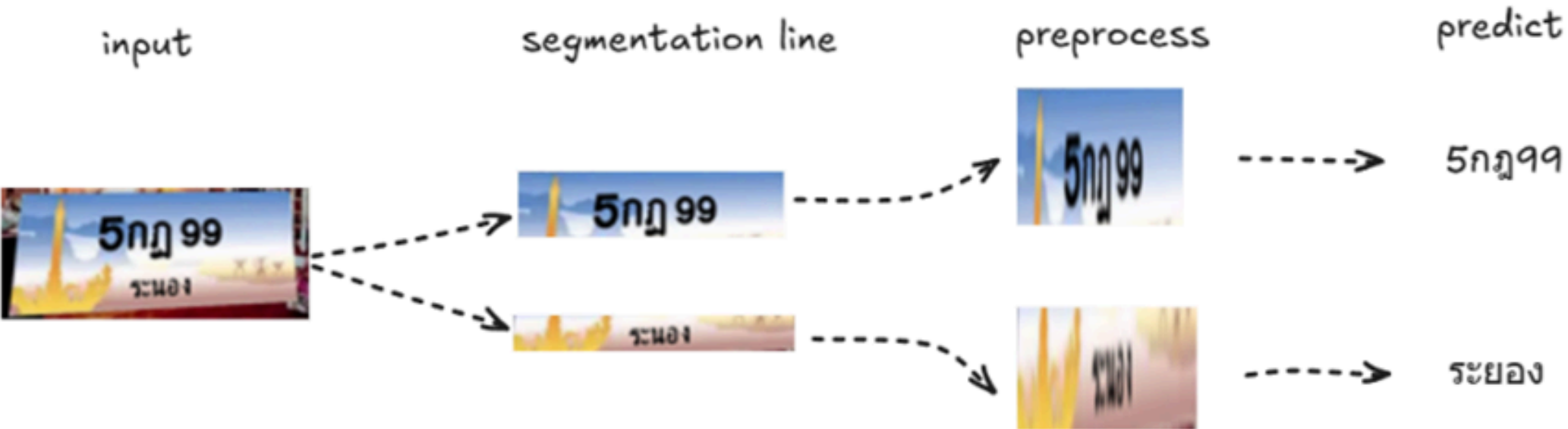
ปัญหาจากการครอบภาพวิดีโอวงจรปิด:

- คุณภาพของภาพที่ได้ค่อนข้างแย่ เนื่องจากโมเดล OCR ส่วนใหญ่ถูกฝึกมาให้ประมวลผลข้อความในบรรทัดเดียว
- หลังจากครอบภาพจากขนาด 1900×1200 ให้เหลือ 140×60 ซึ่งเล็กลงมาก แต่ยังพอประมวลผลได้
- เมื่อแบ่งภาพออกเป็นสองส่วน (หมายเลขทะเบียนและชื่อจังหวัด) แต่ละส่วนจะเหลือขนาด 140×30 ซึ่งเล็กมากและทำให้ข้อมูลสูญหาย
- ต้องปรับขนาดภาพเป็น 224×224 เพื่อให้เข้ากับโมเดล แต่กระบวนการนี้ทำให้คุณภาพภาพลดลงอย่างมาก ทำให้ภาพที่ได้ใช้งานแทบไม่ได้



Methodology

Old method



New method



ข้อดีและข้อเสียระหว่างรูปแบบการเรียนรู้แบบเก่าและแบบใหม่

- ภาพที่มีความละเอียดสูง โมเดลที่ใช้วิธีประมวลผลแบบบรรทัดเดียว (single-line processing) ตามวิธีดั้งเดิมสามารถให้ผลการคาดการณ์ที่ดีได้
- โมเดลที่ใช้วิธีประมวลผลหลายบรรทัด (multi-line processing) มีความเหมาะสมสำหรับงานเฉพาะทาง เช่น การอ่านป้ายทะเบียนรถ โดยไม่จำเป็นต้องใช้หลายโมเดลในการประมวลผลข้อมูล
- โมเดลแบบ multi-line ยังคงต้องการชุดข้อมูลจำนวนมากเพื่อปรับแต่ง (fine-tuning) ให้เหมาะสมกับการใช้งานในแต่ละกรณี



Any Questions?